

LAB 01 - CIFRA DE VIGENÈRE

Mateus Lima Rodrigues - 536334

Introdução:

Esse relatório detalha o desenvolvimento do laboratório 1 da cadeira de criptografia, cujo objetivo foi implementar um algoritmo com os procedimentos de encriptação e desencriptação usando a cifra de Vigenère e a tabela ascii completa. Para a implementação, usei a linguagem Python, todo o funcionamento está no arquivo "[vigenere.py](#)" e os casos de teste estão no arquivo "testes.txt", eles podem ser mudados para o teste que desejar, apenas precisa seguir a norma de "Descricao do Teste;Texto Claro;Chave"

Desenvolvimento:

def encriptar(texto_claro, chave):

Essa é a primeira função do código, ela vai ser responsável por encriptar usando um texto claro como entrada e a chave, ambos fornecidos diretamente do arquivo "testes.txt". O funcionamento dela é bem simples, primeiramente vai ter uma verificação se o texto ou a cifra são vazios, caso aconteça, o caso vai ser considerado como um erro diretamente a criptografia não vai ser feita, explicarei melhor nos casos de teste, depois disso criamos uma variável vazia para representar o texto encriptado e outra para guardar o tamanho da chave, então usamos um laço de repetição para percorrer o texto claro por completo, aqui vamos usar a função `ord()` para transformar um caractere no valor número que ele representa em ascii, então pegamos o valor correspondente do texto e da chave, somamos os dois valores e adicionamos uma garantia que sempre estará na faixa de 0-256 simplesmente fazendo a operação $\% 256$ ao final ficando com a fórmula " $\text{valor_encriptado} = (\text{valor_letra} + \text{valor_chave}) \% 256$ ", depois de tudo só precisamos adicionar esse valor à variável do texto encriptado e retornar

```
def encriptar(texto_claro, chave): # aqui vamos encriptar usando um texto claro e uma cifra, que podem
    if not texto_claro:
        return ("Não foi possível realizar a encriptação. O TEXTO CLARO é vazio")
    if not chave:
        return ("Não foi possível realizar a encriptação. A CHAVE é vazia")

    texto_encriptado = ''
    tamanho_chave = len(chave)

    for i in range(len(texto_claro)):
        valor_letra = ord(texto_claro[i]) # deus abençoe o inventor da função ord
        valor_chave = ord(chave[i % tamanho_chave]) # usando para a chave voltar ao começo quando acabar
        valor_encriptado = (valor_letra + valor_chave) % 256 # matematica para descobrir o número na tabela
        texto_encriptado += chr(valor_encriptado) # deus abençoe o inventor da função chr
    return texto_encriptado
```

def descriptar(texto_encryptado, chave)

Para essa função, temos os parâmetros do texto encryptado da função anterior, e a mesma chave que vai ser usada na operação, aqui começamos fazendo a verificação da mesma forma, se o texto encryptado ou a chave são vazios, em seguida criamos a variável contrária da função anterior, na encryptação criamos o “texto_encryptado” e aqui vamos criar o “texto_claro” para guardar os valores descriptados, fazemos outro laço, dessa vez percorrendo o texto_encryptado, fazemos as operações de transformar os caracteres do texto e da chave em decimais com a função ord() novamente, e em seguida vamos para a função principal que vai subtrair os dois valores, + 256 para garantir que não seja negativo e em seguida % 256 para garantir que esteja dentro da faixa de ascii, então ficamos com a seguinte fórmula “valor_descriptado = (valor_letra - valor_chave + 256) % 256”, então com o resultado do valor, ele é inserido na variável do texto claro e retornado ao final

```
def descriptar(texto_encryptado, chave):
    if not texto_encryptado:
        return ("Não foi possível realizar a descriptação. O TEXTO ENCRYPTADO é vazio")
    if not chave:
        return ("Não foi possível realizar a descriptação. A CHAVE é vazia")

    texto_claro = ''
    tamanho_chave = len(chave)

    for i in range(len(texto_encryptado)):
        valor_letra = ord(texto_encryptado[i])
        valor_chave = ord(chave[i % tamanho_chave])
        valor_descriptado = (valor_letra - valor_chave + 256) % 256 # matematica para reverter a
        texto_claro += chr(valor_descriptado)
    return texto_claro
```

def converte_decimal(texto)

Essa é uma função extra, resolvi fazer só porque percebi alguns problemas com a impressão de caracteres com a tabela do ascii, pelo motivo dos 32 primeiros caracteres serem de controle, e alguns da tabela ascii estendida não terem impressão, então essa função serve para pegar esses caracteres que não podem ser mostrados e imprimir da forma decimal, estava fazendo alguns testes no desenvolvimento e para mim ficou muito estranho o fato de alguns caracteres ficarem invisíveis na impressão, então dessa forma o problema foi resolvido, também virou uma boa ferramenta de “debug” no desenvolvimento, pois me permitiu ver exatamente o decimal que gerava com o caractere [i] do texto com o da chave, então o que ele faz é simplesmente pegar o texto que ele recebe, quebrar cada caractere em um laço, fazer a operação de ord() para pegar o decimal que representa aquele caractere, transformar em uma string usando str() e adicionando a string em uma lista usando .append(), no fim todos serão separados usando um espaço

```
def converte_decimal(texto):
    # alguns caracteres da tabela ascii completa não podem ser impressos
    # os 32 primeiros de controle e a ascii extendida,
    # então essa parte é pra fazer algo com esses resultados
    if not texto:
        return ("Não foi possível realizar a conversão. O TEXTO é vazio")
    lista = []

    for caractere in texto: # percorrendo tds os caracteres do texto e faz op
        valor_decimal = ord(caractere)
        string_decimal = str(valor_decimal)
        lista.append(string_decimal)
    texto_final = ' '.join(lista) # junta tudo e separa com um espaço, então
    return texto_final
```

def ler_teste()

Essa é a função menos emocionante, vai ser responsável por exibir uma pequena interface mostrando os resultados dos testes e também vai fazer a leitura do arquivo de testes, inclusive, vale ressaltar que tem uma exceção caso o arquivo não seja encontrado, então se isso acontecer apenas altere o nome do caminho do arquivo na linha 58.

```
def ler_teste():
    print("----- BEM-VINDO À CIFRA DE VIGENERE.PY -----")
    try:
        with open('../Criptografia/lab1/testes.txt', 'r', encoding='utf-8') as testes: # Abre o arquivo de testes
            for i, linha in enumerate(testes):
                if i == 0: continue # pulando o cabeçalho (linha 0)
                partes = linha.strip().split(';')
                if len(partes) == 3: # garantia q cada linha tem 3 partes
                    descricao, texto_claro, chave = partes
                    print(f"\n--- Teste : {descricao} ---") # descricao do teste
                    print(f"Texto Claro : '{texto_claro}'") # texto claro do teste
                    print(f"Chave : '{chave}'") # chave do teste

                    texto_encryptado = encriptar(texto_claro, chave) # passando o texto claro para a chave
                    texto_desencryptado = desencryptar(texto_encryptado, chave) # passando o texto encryptado para a chave
                    print(f"Texto Encryptado : '{texto_encryptado}'") # print do texto (encryptado)
                    print(f"Representação Decimal : '{converte_decimal(texto_encryptado)}'") # print do texto (decimal)
                    print(f"Texto Desencryptado : '{texto_desencryptado}'") # texto depois de desencryptado

                    if texto_claro == texto_desencryptado: # prova final para testar se funciona
                        print("Resultado : SUCESSO")
                    else:
                        print("Resultado : FALHA")
    except FileNotFoundError:
        print("\nArquivo 'testes.txt' não encontrado, tente mudar o caminho!!")
```

Testes:

Para essa parte de testes, separei 6 para testar alguns casos, alguns que foram sugeridos no documento e outros com casos simples e forçando erros:

- Descrição do Teste; Texto Claro; Chave: Esse é o cabeçalho, representa o modelo de como fazer um teste, primeiro a descrição do que vai realizar no teste, o texto claro e em seguida a chave, tem uma parte no código que faz essa primeira linha ser ignorada, então não tente fazer um teste na primeira linha (if i == 0: continue # pulando o cabeçalho (linha 0))
- Teste Simples; Criptografia; UFC: Esse é um teste simples, contendo um texto e chave simples
 - **--- Teste : Teste Simples ---**
 - **Texto Claro : 'Criptografia'**
 - **Chave : 'UFC'**
 - **Texto Encriptado : 'ϣ¬Å⁰²¹/₄α»¬α'**
 - **Representação Decimal : '152 184 172 197 186 178 188 184 164 187 175 164'**
 - **Texto Desencriptado : 'Criptografia'**
 - **Resultado : SUCESSO**
- Teste de Texto Vazio;; 12345: Esse é um teste para forçar o erro do texto vazio, fiz a implementação de um jeito que em qualquer caso de vazio, o código vai retornar um texto no modelo "Não foi possível realizar x. O y é vazio", então ao final da implementação, se o texto for vazio, ele vai comparar o texto claro original, nesse caso vazio, e o texto desencriptado, na implementação "Não foi possível realizar x. O y é vazio", acusando erro
 - **--- Teste : Teste de Texto Vazio ---**
 - **Texto Claro : ''**
 - **Chave : '12345'**
 - **Texto Encriptado : 'Não foi possível realizar a encriptação. O TEXTO CLARO é vazio'**
 - **Representação Decimal : '78 227 111 32 102 111 105 32 112 111 115 115 237 118 101 108 32 114 101 97 108 105 122 97 114 32 97 32 101 110 99 114 105 112 116 97 231 227 111 46 32 79 32 84 69 88 84 79 32 67 76 65 82 79 32 233 32 118 97 122 105 111'**
 - **Texto Desencriptado : '±<ì1>7í<:BA°B0;î?1,;7G-=î/î192@6<?0μ°;ùîî ""îîC-E8='**
 - **Resultado : FALHA**
- Teste de Erro na Chave; UFC-Quixadá: Mesmo caso do teste anterior, aqui vamos testar se uma chave é vazia, nesse caso, eu poderia fazer com que a implementação retornasse o próprio texto claro, uma vez que não tem chave para realizar a implementação, mas no contexto aplicado, onde eu devo criptografar uma mensagem, preferi colocar um caso de erro, uma vez que a mensagem não pode sair do mesmo jeito que entrou
 - **--- Teste : Teste de Erro na Chave ---**
 - **Texto Claro : 'UFC-Quixadá'**

- **Chave :** "
 - **Texto Encriptado :** 'Não foi possível realizar a encriptação. A CHAVE é vazia'
 - **Representação Decimal :** '78 227 111 32 102 111 105 32 112 111 115 115 237 118 101 108 32 114 101 97 108 105 122 97 114 32 97 32 101 110 99 114 105 112 116 97 231 227 111 46 32 65 32 67 72 65 86 69 32 233 32 118 97 122 105 97'
 - **Texto Desencriptado :** 'Não foi possível realizar a desencriptação. A CHAVE é vazia'
 - **Resultado :** FALHA
- **Chave Maior que o Texto;TESTE;UMA CHAVE BEM MAIOR:** Esse é um caso para forçar a condição do texto acabar mais rápido que a chave, mais comumente aconteceria o contrário e a chave acabaria primeiro e voltaria para o começo
 - **--- Teste : Chave Maior que o Texto ---**
 - **Texto Claro :** 'TESTE'
 - **Chave :** 'UMA CHAVE BEM MAIOR'
 - **Texto Encriptado :** '©t'
 - **Representação Decimal :** '169 146 148 116 136'
 - **Texto Desencriptado :** 'TESTE'
 - **Resultado :** SUCESSO
- **Caracteres Especiais e Números;Senha@123!;segredo:** Esse é um teste para mostrar ao máximo a diferença entre esse modelo e a cifra de vigenère original
 - **--- Teste : Caracteres Especiais e Números ---**
 - **Texto Claro :** 'Senha@123!'
 - **Chave :** 'segredo'
 - **Texto Encriptado :** 'ÆÊÕÚÆ¤ ¥'
 - **Representação Decimal :** '198 202 213 218 198 164 160 165 152 136'
 - **Texto Desencriptado :** 'Senha@123!'
 - **Resultado :** SUCESSO
- **Texto com Acentuação;Ceará é campeão!;Quixadá:** Parecido com o caso anterior, mas esse foca mais em mostrar que a acentuação vai transformar o caractere em outro, sendo é != e
 - **--- Teste : Texto com Acentuação ---**
 - **Texto Claro :** 'Ceará é campeão!'
 - **Chave :** 'Quixadá'
 - **Texto Encriptado :** 'ÚÊêBÊqØÊâÑÉÄÀ'
 - **Representação Decimal :** '148 218 202 234 66 132 202 113 216 202 229 209 201 196 192 150'
 - **Texto Desencriptado :** 'Ceará é campeão!'
 - **Resultado :** SUCESSO